Agent Postmortem Generator

Part 1 - Canonical Trace Model

Module Design Document

1. Title Page

Project Name: Agent Postmortem Generator
Part Number: 1
Module Name: Canonical Trace Model
One-line Purpose: Define the project-owned, framework-agnostic normalized trace representation that every downstream analysis step can trust.

2. Executive Overview

The **Canonical Trace Model** is the stable internal contract that sits immediately after normalization and immediately before analysis. It converts a validated, framework-neutral execution trace into a project-owned document model that preserves chronology, structure, evidence, and provenance without embedding any framework-specific behavior.

This module exists because the rest of the system should never need to understand LangGraph, CrewAI, OpenAI Agents SDK, AutoGen, or any future adapter format. Those concerns belong at the adapter and normalizer boundaries. Once a trace enters this module, it must already be expressed in canonical terms: runs, nodes, events, evidence, and provenance.

The module is the heart of the deterministic pipeline:

```
Raw trace
  -> Parser
  -> Validation
  -> Normalizer
  -> Canonical Trace Model
  -> Analysis Engine
  -> Postmortem Generator
  -> Output Layer
```

The design goal is not to model every possible agent framework. The goal is to define the smallest useful internal language for agent execution evidence so that failure analysis can be deterministic, testable, and explainable.

3. Problem Statement

AI agent traces are inconsistent by default. One framework emits tool call objects, another emits spans, another emits nested messages, and another logs everything as loosely structured JSON. Some traces are complete. Some are partial. Some contain retries, cancellations, or missing timestamps. Some are ordered. Some are not. Some are rich in content but poor in structure. Humans, naturally, then ask the system to "just figure out what happened," as if ambiguity were a feature.

This module solves the problem of **representing execution facts in a stable, normalized form** that downstream logic can analyze without re-learning each framework's quirks.

Why this problem is hard

- Agent traces are often hierarchical, but the hierarchy is implicit.
- The same logical action may appear as multiple raw events across retries or substeps.

- Timestamps may be missing, duplicated, skewed, or only partially ordered.
- Evidence may be spread across messages, tool calls, logs, and errors.
- Framework objects often mix transport details with semantic meaning.
- Downstream logic becomes unmaintainable if it has to branch on raw source formats.

Why the MVP needs this module

Without a canonical trace model, the project collapses into a pile of adapter-specific conditionals. That destroys the main product promise: deterministic, evidence-based postmortems from normalized traces. The MVP needs one stable internal schema so that the analysis engine can focus on causality and failure reconstruction rather than input cleanup.

4. Goals

What success looks like

Goal	What it means in practice
Stable contract	The same normalized trace always produces the same canonical model.
Framework agnostic	No downstream code depends on source framework objects or raw adapter shapes.
Evidence preserving	Important facts, payloads, and provenance survive normalization.
Deterministic	Ordering, identifiers, and validation rules produce repeatable results.
Testable	The model can be validated with fixtures and structural assertions.
Maintainable	The schema remains small enough for a small team to evolve safely.

What this module must achieve

- Define a canonical schema for a trace run and its execution facts.
- Preserve the minimum evidence required for analysis and postmortem generation.
- Encode structure, chronology, and provenance without interpreting blame or root cause.
- Provide stable versioning for future schema evolution.
- Reject malformed internal states rather than silently papering over them.

5. Non-Goals

This module explicitly does **not**:

- Parse raw framework traces.
- Infer root cause, blame, confidence, or remediation.
- Render reports, diagrams, or PDFs.
- Call external services or LLMs.
- Store operational state, caches, or user preferences.
- Implement framework-specific adapter logic.
- Perform heuristic "repairs" that change trace meaning.
- Support every trace feature from every ecosystem on day one.

Later-version responsibilities

The following belong to future work, not the MVP:

- OpenTelemetry-native ingestion.
- Rich event visualization models.
- Multi-run comparison data structures.
- Drift detection representations.
- Token accounting and cost analysis fields if they require broader ecosystem support.
- PDF-optimized presentation layers.

6. Responsibilities

What this module owns

- The canonical trace schema.
- Entity identity rules.
- Trace-level and entity-level invariants.
- Stable serialization shape for the canonical document.
- Provenance references linking canonical facts back to source records.
- Versioning of the canonical schema.
- Deterministic ordering semantics.

What it must never own

- Raw adapter parsing.
- Source-framework business logic.
- Failure interpretation.
- Output formatting.
- Recommendation generation.
- LLM-based enrichment.
- Persistence strategy.
- Cross-run aggregation.

- Cloud or network concerns.

If a decision requires knowledge of a source framework's behavior, that decision belongs upstream in the normalizer.

7. Inputs

The canonical model consumes **normalized trace data**, not raw framework objects.

Expected inputs

The normalizer should supply a structure that already expresses:

- One execution run.
- Ordered events.
- Logical execution nodes.
- Parent-child relationships where available.
- Structured evidence payloads.
- Provenance information pointing to the origin of each fact.
- Validation status from earlier pipeline stages.

Validation rules

Rule	Expectation	Rationale
Schema version present	Required	Prevents ambiguous interpretation of the document.
Stable document identity	Required	Enables fixture testing and reproducible analysis.
Unique entity identifiers	Required	Prevents broken references and duplicate merges.
Valid parent references	Required for non-root nodes	Preserves trace topology.
Deterministic ordering key	Required	Keeps analysis repeatable.
Explicit unknowns	Required when data is missing	Avoids invented meaning.
No circular parentage	Required	Prevents invalid execution graphs.
Source provenance	Required for evidence-bearing fields	Preserves auditability.

Assumptions

- The normalizer has already resolved framework-specific shapes into project-owned concepts.
- The model should receive one coherent trace run per document in the MVP.
- Missing optional details are acceptable if they are explicitly marked as unknown or absent.
- The model does not need to guess missing semantics.

Malformed input handling

The canonical model should fail fast on structural violations:

- Duplicate IDs.
- Broken parent links.
- Cycles in the execution tree.
- Conflicting status fields.
- Missing required ordering fields.
- Unknown enum values without an explicit fallback category.
- Evidence records that cannot be tied to a provenance source.

The module should prefer a hard validation error over "best effort" interpretation. Best effort belongs earlier in the pipeline, where source-specific context still exists.

8. Outputs

Exact outputs

The module produces a **canonical trace document** that downstream code can consume as the single source of truth for analysis.

A valid output includes:

- A top-level trace document.
- Run metadata.
- Ordered canonical events.
- Execution nodes and their relationships.
- Evidence records or evidence references.
- Provenance metadata.
- Validation notes, when non-fatal.

Data artifacts

Artifact	Purpose	Notes
Canonical trace document	Primary output	Immutable internal representation.
Stable serialized JSON form	Debugging and fixtures	Must preserve deterministic ordering.
Schema version marker	Compatibility	Needed for migrations later.
Validation report	QA and rejection logic	Includes structural errors and warnings.

Human-readable artifacts

This module should not generate reports, summaries, or narrative output. The closest human-readable artifact is a stable debug serialization of the canonical document for fixtures, diffing, and test inspection.

9. Internal Design

The module should be implemented as a **small domain package**, not as a service, not as a framework, and not as a dumping ground for utility functions.

Main subcomponents

```
Canonical Trace Model
  +-- Schema definitions
  +-- Identity and reference rules
  +-- Entity containers
  +-- Invariant validation
  +-- Deterministic serialization
  +-- Provenance mapping
```

1. Schema definitions

Defines the structure of the canonical document and the allowed entity types. These are project-owned types, not framework types.

2. Identity and reference rules

Defines how IDs are created, checked, and referenced. IDs must be opaque and stable within a document.

3. Entity containers

Holds the document, run metadata, nodes, events, evidence, and issues in a normalized layout.

4. Invariant validation

Ensures the model is internally coherent before analysis ever sees it.

5. Deterministic serialization

Produces a stable JSON representation for fixtures, regression tests, and debugging.

6. Provenance mapping

Connects each important canonical fact back to the original source location.

Workflow

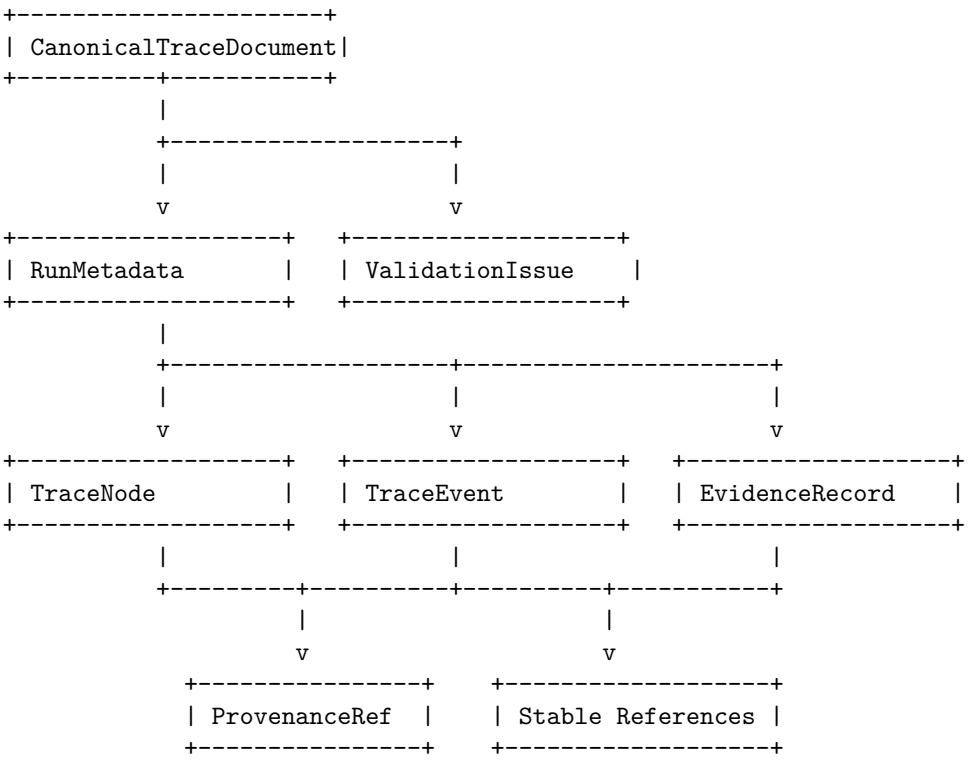
```
Normalized trace input
        |
        v
  Build canonical entities
        |
        v
  Assign stable IDs and sequence order
        |
        v
  Attach provenance references
        |
        v
  Validate structural invariants
        |
        v
  Freeze / emit canonical document
        |
        v
   Analysis engine consumes it
```

Important internal decisions

1. **The model is immutable after construction.**
   Downstream analysis should not mutate the source of truth.
2. **Ordering must be explicit.**
   Do not rely on map iteration, object insertion quirks, or hash ordering.
3. **Unknown values must be represented, not guessed.**
   The model should distinguish "missing," "unknown," and "not applicable."
4. **Causality is not precomputed here.**
   The model should preserve facts and structure, but not pretend to know root cause.

5. **Provenance is a first-class concern.**
   Every meaningful fact should be traceable to where it came from.

Internal dependency graph



10. Data Model

The canonical trace model should be deliberately small but expressive enough to support failure reconstruction.

Core entities

Entity	Responsibility	Key fields
CanonicalTraceDocument	Top-level container	document_id, schema_version, run, nodes, events, evidence, issues

Entity	Responsibility	Key fields
RunMetadata	Describes the execution run	run_id, status, started_at, ended_at, source_format, environment
TraceNode	Represents a logical unit of execution	node_id, kind, parent_node_id, attempt_index, status, timestamps, attributes
TraceEvent	Captures a point-in-time fact	event_id, sequence_index, timestamp, kind, node_id, payload, severity
EvidenceRecord	Stores or references concrete evidence	evidence_id, evidence_type, content_ref, content_summary, provenance
ProvenanceRef	Ties canonical data to source data	source_record_id, source_path, source_file, raw_key_path
ValidationIssue	Records structural problems or warnings	issue_id, severity, field_path, message, source_ref

Key fields and semantics

CanonicalTraceDocument

The document is the immutable root object. It contains everything analysis needs for one run.

RunMetadata

Describes the execution context without embedding framework logic. The source format may be known for provenance, but the rest of the model should not care which adapter created it.

TraceNode

A node is a logical execution unit such as a model call, tool call, agent step, or sub-agent step. Nodes organize the trace into a structure that analysis can reason about.

Suggested core fields:

- `node_id`: opaque identifier.
- `kind`: small controlled vocabulary.
- `parent_node_id`: optional tree relationship.
- `series_id`: optional grouping identifier for retries or repeated attempts.
- `attempt_index`: optional ordering within a series.
- `status`: current outcome of the node.
- `started_at` / `ended_at`: normalized timestamps where available.
- `attributes`: small extension bag for non-core normalized metadata.

TraceEvent

An event is an atomic fact in the run timeline.

Suggested core fields:

- `event_id`: opaque identifier.
- `sequence_index`: deterministic ordering key.
- `timestamp`: normalized point-in-time value when available.
- `kind`: start, end, log, error, message, input, output, retry, cancellation, or similar canonical kind.
- `node_id`: optional attachment to a node.
- `payload`: evidence payload, usually structured JSON or text.
- `severity`: informational, warning, error, or critical.

EvidenceRecord

Evidence records preserve concrete source content that analysis may cite later. This can be inline content for small records or a reference for larger payloads.

ProvenanceRef

Provenance is not an optional nice-to-have. It is what makes the postmortem evidence-based rather than interpretive fiction.

Relationships

```
CanonicalTraceDocument
  -> RunMetadata
  -> TraceNode[]      (one-to-many)
  -> TraceEvent[]     (one-to-many)
  -> EvidenceRecord[] (one-to-many)
  -> ValidationIssue[]

TraceNode
  -> parent_node_id   (optional self-reference)
  -> EvidenceRecord[]  (via references)
  -> TraceEvent[]      (via node_id)
```

```
TraceEvent
  -> node_id          (optional)
  -> EvidenceRecord   (via payload or reference)
  -> ProvenanceRef    (required for meaningful facts)
```

Why this model exists

The analysis engine needs a representation that is: - stable enough for deterministic reasoning, - rich enough to preserve evidence, - narrow enough to stay maintainable, - and generic enough to support future adapters.

That combination is the whole point. Any model that tries to be "universal" by absorbing every framework's peculiarities will become impossible to maintain. Any model that is too small will starve the analysis engine of evidence. This design aims for the narrow middle.

11. Algorithms / Processing Logic

This module should not perform high-level analysis. Its logic is about **construction, validation, and normalization of the canonical representation itself**.

Step-by-step processing

1. Receive normalized trace input.
2. Confirm schema version and top-level identity.
3. Materialize run metadata.
4. Materialize nodes in deterministic order.
5. Materialize events in deterministic order.
6. Bind events to nodes where relationships are known.
7. Attach evidence records and provenance references.
8. Validate structural invariants.
9. Collect non-fatal issues.
10. Emit the immutable canonical document.

Decision rules

Ordering

- If `sequence_index` exists, it wins over incidental input order.
- If timestamps exist, they may assist ordering, but they must not override explicit sequence order unless the schema says so.
- When order is ambiguous, preserve input order and mark the ambiguity explicitly.

Missing data

- Optional fields may be absent.
- Required structural fields may not be invented.
- If a field is semantically unknown, mark it as unknown rather than null unless the schema differentiates them.
- Do not collapse "not observed" into "did not happen."

Retry handling

Retries should not be hidden by deduplication. If multiple attempts exist, represent them as separate nodes or events within the same logical series, with attempt metadata preserved. This keeps failure reconstruction honest.

Failure states

The model should distinguish: - node failure, - event-level error, - run-level failure, - and partial completion.

A single error event does not always mean the entire run failed. The schema should preserve that distinction.

Edge cases

Edge case	Model behavior	Why
Missing timestamps	Accept if sequence order exists	Some traces are partially timestamped.
Duplicate tool outputs	Preserve as distinct events if evidence differs	Lossless representation is safer.
Retry loops	Group with series metadata	Supports failure propagation analysis.
Partial traces	Accept if structural minimum exists	Real traces are often incomplete.
Canceled runs	Explicitly mark canceled status	Avoids confusion with failure.
Nested sub-agents	Use parent-child node relations	Keeps hierarchy readable.
Out-of-order raw input	Reorder only by canonical rules	Guarantees deterministic output.

Deterministic behavior requirements

- Same input must produce same document.
- Same document version must serialize identically for the same logical content.
- Collections must have explicit ordering rules.

- Validation must not depend on runtime randomness, system locale, or hash seeds.
- No implicit merging of semantically distinct events.
- No hidden sampling, ranking, or inference.

12. Tradeoffs

Simplicity vs flexibility

Chosen design: a small core schema plus a limited extension bag.

Why: The project is an open-source developer tool, not a research sandbox. Too much flexibility in the core model creates unstable downstream expectations.

Tradeoff: Some niche framework fields will not map cleanly into the core schema.

Why that is acceptable: Those fields can remain in provenance or normalized attributes. They do not need to pollute the stable core.

Generality vs specificity

Chosen design: specific enough to support agent debugging, not general enough to become a universal trace ontology.

Why: The module exists to explain failed AI agent runs. It does not need to model every distributed system in the world.

Tradeoff: The model may not capture every observability use case.

Why that is acceptable: That belongs to later OpenTelemetry-oriented expansion, not the MVP.

Determinism vs heuristic inference

Chosen design: determinism wins.

Why: The product promise is evidence-based postmortems. If the model invents meaning, the analysis becomes harder to trust.

Tradeoff: Some traces will remain partially ambiguous.

Why that is acceptable: Ambiguity should be surfaced to the analysis engine, not silently erased in the model layer.

Why this design is preferred

This design is boring in the best possible way. It is small, explicit, reproducible, and easy to test. Boring infrastructure tends to survive contact with reality.

Clever models tend to expire in the first messy incident.

13. Risks and Failure Modes

Hidden complexity

The hardest part is not storing data. The hard part is preserving the right amount of structure without accidentally encoding framework-specific assumptions.

Ambiguous cases

- Events with no clear parent node.
- Retries that look like duplicated actions.
- Partial runs where end-state is missing.
- Logs that imply causality but do not prove it.

The model should preserve ambiguity rather than resolving it too early.

Unsupported inputs

The MVP does not need to support: - arbitrary nested schema variants, - multi-run correlation, - live streaming traces, - or framework-specific object graphs.

Those should be rejected or deferred upstream.

Maintenance risks

Risk	Impact	Mitigation
Schema sprawl	High	Keep enums and extension fields small.
Implicit semantics creep	High	Require explicit review for new fields.
Weak provenance	High	Make source references mandatory for meaningful facts.
Over-normalization	Medium	Preserve evidence instead of collapsing detail.
Under-normalization	Medium	Ensure downstream never sees raw framework shapes.

14. Testing Strategy

The canonical trace model should be heavily fixture-tested. This module is a schema contract, so regressions are mostly structural, not algorithmic.

Unit tests

- Validate required fields.
- Reject duplicate IDs.
- Reject cycles in parent relationships.
- Verify deterministic ordering behavior.
- Confirm unknown values are handled explicitly.
- Check version handling.

Integration tests

- Feed normalized fixtures into the model and compare the resulting canonical document to expected snapshots.
- Verify that downstream analysis receives the exact same shape across repeated runs.

Fixture-driven tests

Use representative fixtures for: - a clean successful run, - a single tool-call failure, - a retrying run, - a partial trace, - a canceled run, - a nested sub-agent trace, - a malformed trace with structural errors.

Fixture tests should be stable and human-readable. If a fixture becomes impossible to understand, it will also become impossible to maintain.

Regression protection

- Snapshot the serialized canonical document.
- Lock down ordering behavior.
- Add tests whenever a new canonical field is introduced.
- Preserve backward compatibility in a controlled way through schema versioning.

15. Implementation Checklist

1. Define the top-level canonical document shape.
2. Define versioning and identity rules.
3. Define the core entity types and allowed enums.
4. Define provenance structures for source linking.
5. Define ordering rules for events and nodes.
6. Define validation invariants and error types.

7. Define serialization rules for stable output.
8. Create fixtures for representative traces.
9. Add snapshot tests for serialized canonical documents.
10. Add negative tests for malformed or ambiguous inputs.
11. Confirm the model remains framework agnostic.
12. Document the schema contract for the normalizer and analysis engine.
13. Review every new field for long-term compatibility before merging.

16. Future Expansion

The following expansions naturally follow from this module and can be added later without breaking the MVP architecture:

- OpenTelemetry span and event mapping into the same canonical vocabulary.
- Multi-run document wrappers for cross-run comparison.
- Richer provenance indexing for larger traces.
- Optional content deduplication for repeated payloads.
- Stable export formats for graph visualization.
- Compatibility migration helpers for future schema versions.
- Additional normalized evidence types once new adapters require them.

These should only be added when there is a concrete downstream need, not because the schema "could" support them.

17. Final Recommendation

This design is **MVP-ready** if the implementation stays strict about scope.

The module should be kept as a small, immutable, versioned schema package with strong validation and explicit provenance. That gives the analysis engine a reliable foundation without inviting premature abstraction.

Recommended simplifications to keep

- Keep the core entity set small.
- Keep enums intentionally narrow.
- Keep provenance mandatory for meaningful facts.
- Keep the model immutable after creation.
- Keep framework-specific details out of the core schema.

Recommended simplifications still worth preserving

- No optional "smart" inference in the model layer.
- No derived root-cause fields.
- No adapter shortcuts that bypass validation.

- No general-purpose observability ontology in the MVP.

If the canonical trace model remains disciplined, the rest of the system gets simpler, safer, and much easier to trust. That is the whole game.